Underworld and Docker (part 2)

Julian Giordani[1], and **Louis Moresi**[2]

[1]University of Sydney, [2]Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193341

Keywords AuScope UW Cloud

By the way, before you read this post, catch up with how we use `docker` by reading part 1

Docker allows us to distribute pre-built applications which are hosted in a virtual machine and are therefore platform independent. This simplifies things for us (only one platform we need to support) and for the users (you get a universal binary with complete reproducibility across platforms). However, if you are not running on a native linux machine, you have to understand the way docker works before being able to access the binary.

Docker allows us to provide a single image to our users but not a consistent user experience. `Kitematic` is a Docker-supported user interface which allows users to discover and run Docker containers and interact with them in a predictable manner. With `Kitematic` you can download and run the Underworld containers through a gui which provides clickable links to the notebook server.

At present, `Kitematic` is a mac application and an alpha version exists for windows (and we can already tell you how to launch a browser directly into the notebooks on linux).

1. Getting started

Launching `Kitematic` for the first time brings up an app-store-like list of available containers. Underworld2 is available through the Docker hub and so can be discovered by searching:

PUBLISHED Sep 15, 2015
ARTICLE UWTN 2015-003
VERSION 1.0.0

ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/underworld-and-docker-part-2/

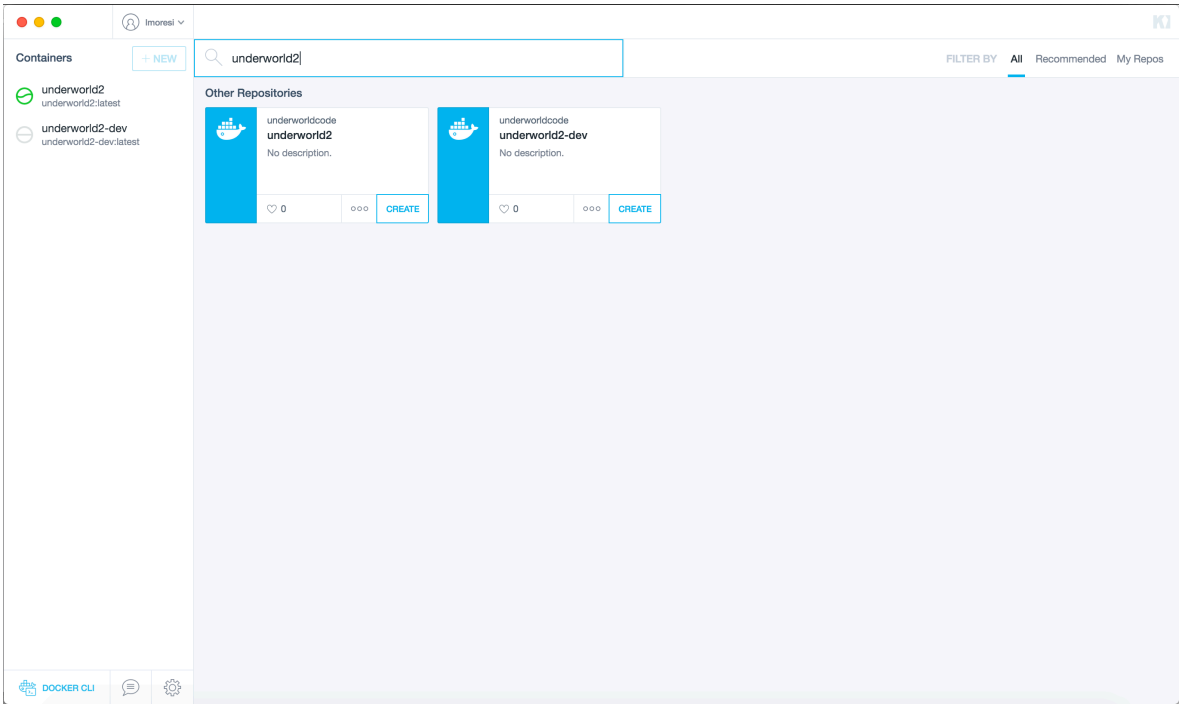


Figure 1: Run Kitematic and search for Underworld2

Clicking `Create` on the *Underworld2* container will automatically download it and launch into the default configuration. This container is tested code from the release branch. If you want a specific version, check out the tags (at the moment we don't have a history of past releases though !). The *Underworld2-dev* container is a rolling build which is updated from the development branch whenever there are changes. You can rename your container from the General / Settings tab which is not such a bad idea if you are using the latest build of a development branch because the container, once created, is an immutable snapshot of that version of the code.

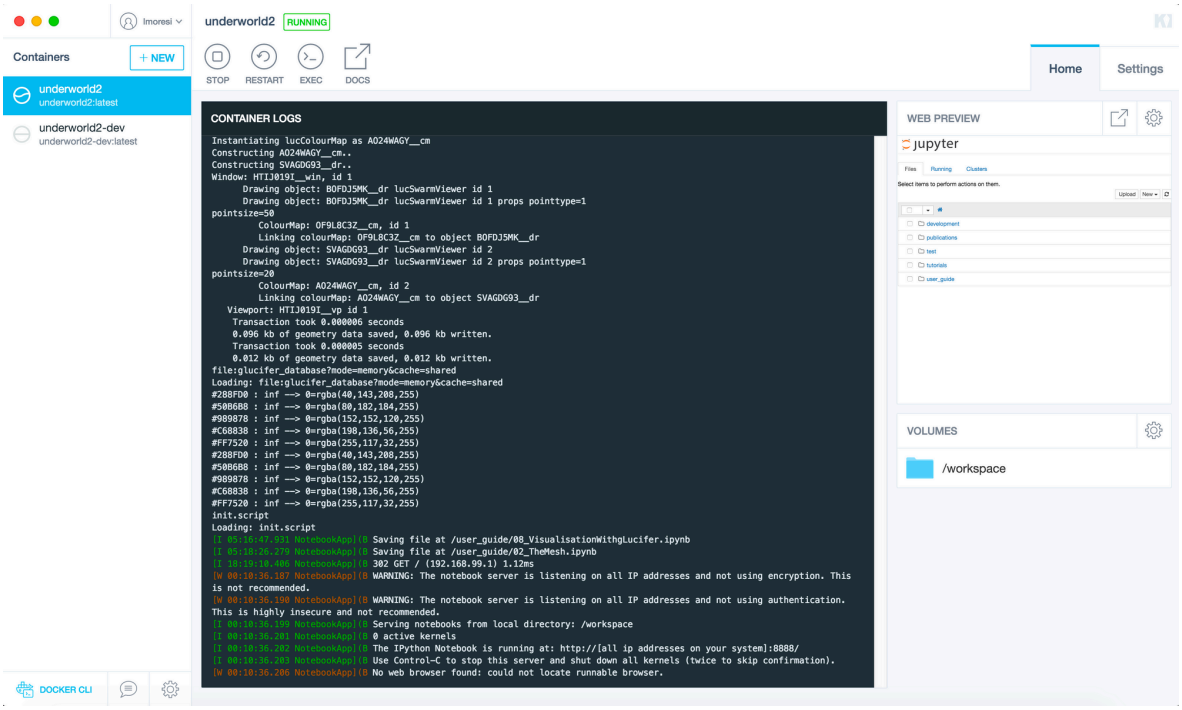


Figure 2: *Once Underworld is running, you will see a web-preview under the Home tab. In the windows alpha version this will be blank, but clicking on it will open the correct container in your browser.*

The underworld container is set to serve up the User Guide (notebooks) by default. If you click on the web-preview it will launch your default browser with the appropriate IP address and port (you can change the port in the settings if you want).

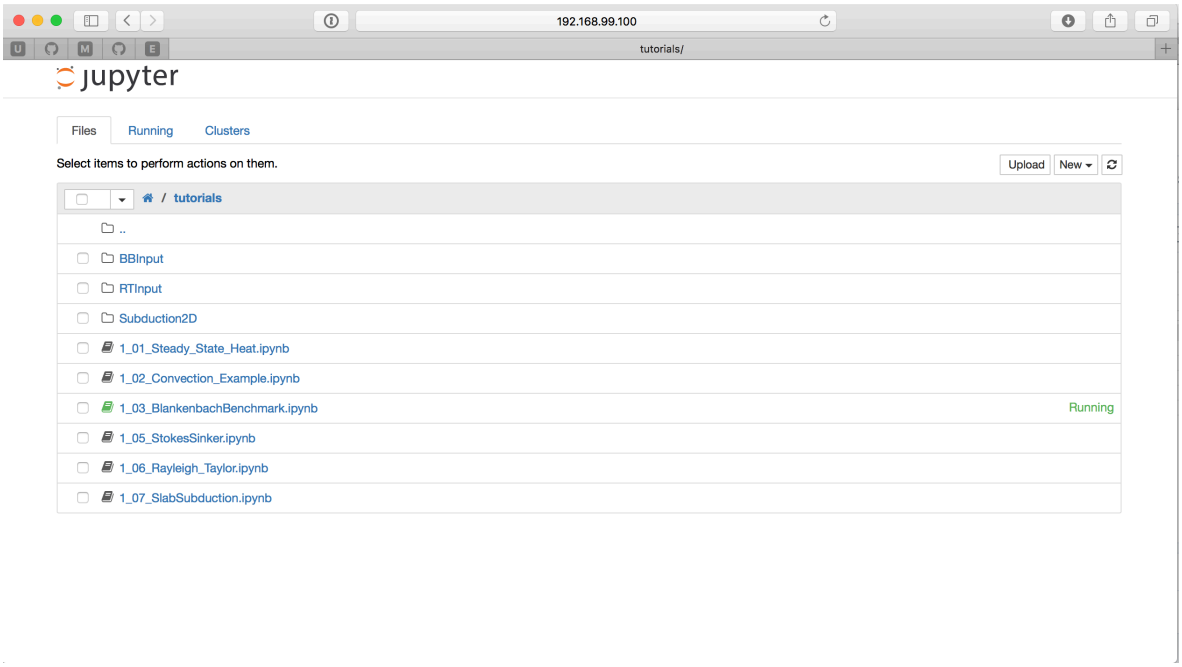


Figure 3: The web browser opens to the default IP address and port for the virtual machine running the underworld notebooks.

When you run the notebooks in your browser, the virtual machine will show the output of the code in the main window (see Figure 2 above). Any output and changes you make to the notebooks - and any new notebooks - will appear within the virtual environment. The simplest way to access the machine itself is also via the web browser. You can start a new terminal from the homepage (Figure 3) of the notebook session. This brings into a shell in the root directory of the virtual machine running underworld. Try running `uname -a`, it won't look very mac-like.

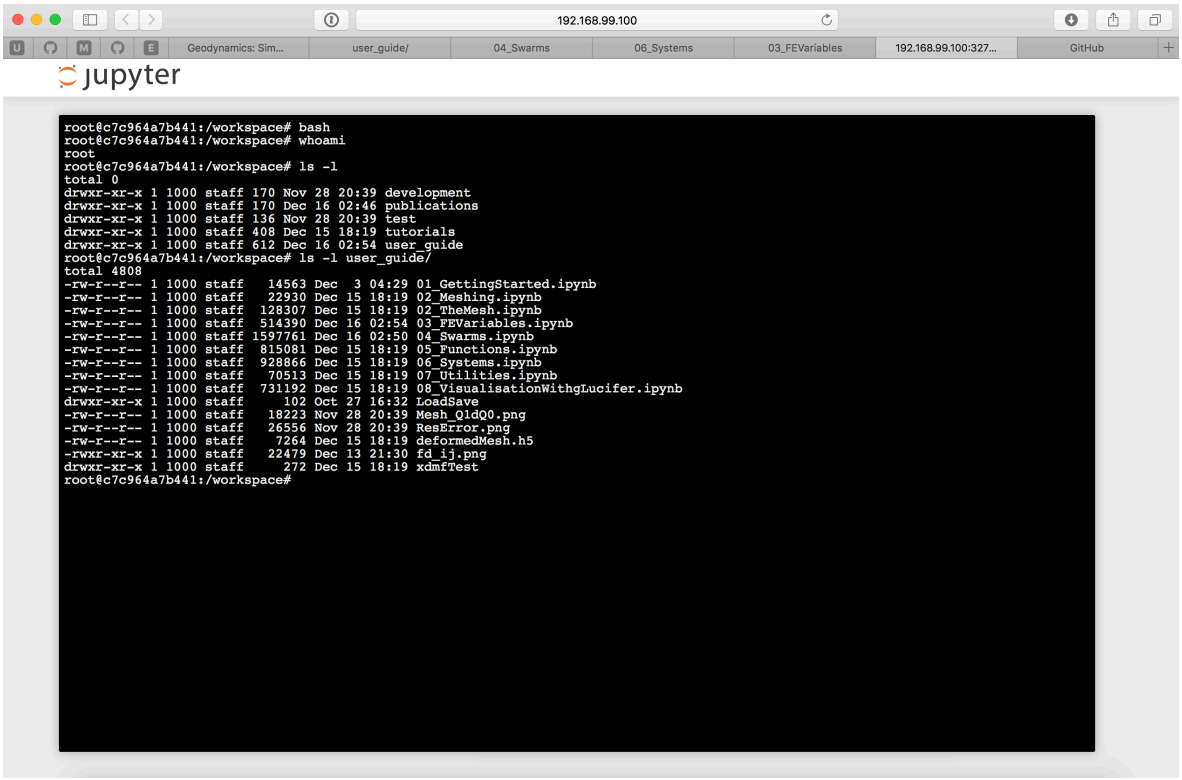
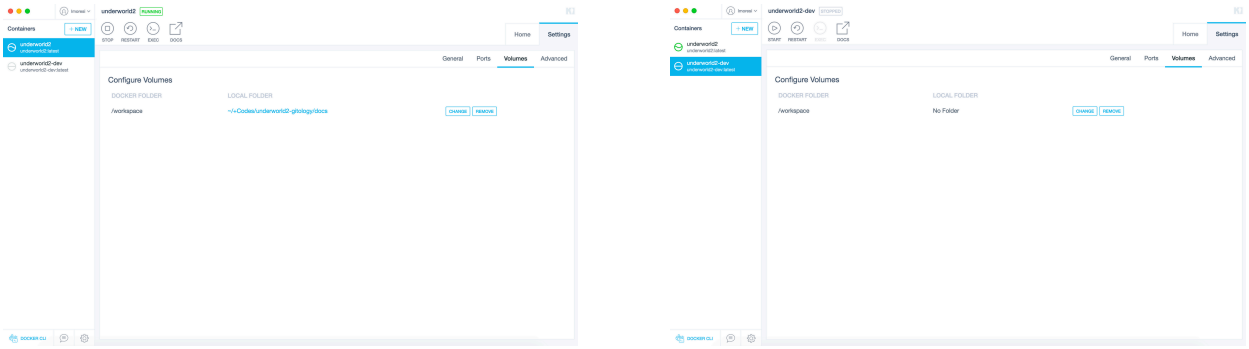


Figure 4: Launching a new terminal from the notebook home screen is a quick way into the virtual machine.

2. How to work with your own notebooks using `Kitematic`

It does not make sense to keep all of your own work inside the virtual environment. If you want to manage your own data and keep notebooks synchronised / versioned in your own repository, and collaborate with others using shared space, then you can connect this space through to the virtual environment visible to the underworld machine.

This is done through the settings tab of the container



(a) (b)

Figure 5: Allowing the outside world into the container by mounting a local directory.

When the workspace is set to `no folder`, all data is written within the container itself. When you choose a folder in the outside world, this is the default workspace which the underworld2 environment uses. Notebooks can be run from within that directory or its sub-directories and can also be accessed from the standard filesystem. If you have data which you want to share between different containers, you can point them all to some common space. The usual caveats apply though if you have several machines trying to use common files or write to a common directory.

You can access the contents of the `/workspace` volume to save any changes you make within the container because `Kitematic` tries to mount these locally in a default location see https://kitematic.com/docs/managing-volumes/ for details. You need to click the volume first to prompt the program to do this.

3. Caution !!

Some actions within Kitematic have unexpected side effects. Renaming a container actually will stop and restart it which means it reverts to the original image and you will lose any changes you have made within the container.